

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 12_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of non-negative integers, the task is to find the minimum number of elements such that their sum should be greater than the sum of the rest of the elements of the array.

Example:

Input: arr[] = [3 , 1 , 7 , 1]

Output: 1

Explanation: Smallest subset is {7}. Sum of this subset is greater than the sum of all other elements left after removing subset {7} from the array

Input: arr[] = [2 , 1 , 2]

Output: 2

Explanation: Smallest subset is {2 , 1}. Sum of this subset is greater than the sum of all other elements left after removing subset {2 , 1} from the array

Input Format

The first line contains an Integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single integer representing the minimum number of elements whose sum is greater than the sum of the remaining elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3 1 7 1

Output: 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
void sort(int arr[], int n) {  
    for (int i = 0; i < n-1; i++)  
        for (int j = i+1; j < n; j++)  
            if (arr[i] < arr[j]) {  
                int t = arr[i];  
                arr[i] = arr[j];  
                arr[j] = t;  
            }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```

int arr[25];
int total = 0;

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
    total += arr[i];
}

sort(arr, n);

int sum = 0, count = 0;

for (int i = 0; i < n; i++) {
    sum += arr[i];
    total -= arr[i];
    count++;

    if (sum > total)
        break;
}

printf("%d", count);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an array `prices[]` of `n` different candies, where `prices[i]` represents the price of the `i`th candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to `k` other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

Note: In both cases, you must take the maximum number of free candies possible during each purchase.

Examples:

Input: prices[] = [3, 2, 1, 4], k = 2

Output: [3, 7]

Explanation: For minimum cost, buy the candies costing 1 and 2. for maximum cost, buy the candies costing 4 and 3.

Input: prices[] = [3, 2, 1, 4, 5], k = 4

Output: [1, 5]

Explanation: For minimum cost, buy the candy costing 1. for maximum cost, buy the candy costing 5.

Input Format

The first line contains an Integer n, representing the number of candies.

The second line contains n space-separated integers, representing the prices of the candies.

The third line contains an integer k, representing the number of free candies per purchase.

Output Format

The output prints two integers separated by a space:

minCost maxCost

representing the minimum and maximum money needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3 2 1 4

2

Output: 3 7

Answer

// You are using Java

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        int[] arr = new int[n];
```

```
        for (int i = 0; i < n; i++)  
            arr[i] = sc.nextInt();
```

```
        int k = sc.nextInt();
```

```
        Arrays.sort(arr);
```

```
        int minCost = 0, maxCost = 0;
```

```
        int i = 0, j = n - 1;
```

```
        // Minimum cost  
        while (i <= j) {  
            minCost += arr[i];  
            i++;  
            j -= k;  
        }
```

```
        i = 0;  
        j = n - 1;
```

```
        // Maximum cost  
        while (i <= j) {  
            maxCost += arr[j];  
            j--;  
            i += k;  
        }
```

```
        System.out.print(minCost + " " + maxCost);
```

```
    }  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Given a fraction represented by two integers – a numerator n and a denominator d – express the fraction as a sum of unique unit fractions (fractions with numerator 1). This representation is known as the Egyptian Fraction.

Write a program to ensure that the given fraction is converted into its Egyptian Fraction form.

Input Format

The input contains two space-separated integers n and d , representing the numerator and denominator of the fraction.

Output Format

The output prints the Egyptian Fraction representation of the given fraction in the form of unit fractions separated by +.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 14

Output: 1/3 + 1/11 + 1/231

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n, d;
    scanf("%d %d", &n, &d);
```

```
    while (n != 0) {
        int x = (d + n - 1) / n;
```

```
printf("1/%d", x);

n = n * x - d;
d = d * x;

if (n != 0)
    printf(" + ");
}

return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is building a data compression feature in her file organizing system. She wants filenames to have no repeated characters and appear in the smallest possible order so users can locate files easily. For example, if a corrupted filename is "cbacdcabc", she wants it cleaned up as "acdb" — which preserves one instance of each letter, keeps relative order, and is the smallest alphabetically.

This function helps ensure filenames are clean, minimal, and alphabetically organized!

Example 1:

Input: s = "bcabc"

Output: "ABC"

Example 2:

Input: s = "cbacdcabc"

Output: "acdb"

Input Format

The first line of input contains a string num representing a non-negative integer.

The second line contains an integer k representing how many digits to remove.

Output Format

The output prints the string representing the smallest possible number after removing exactly k digits.

Leading zeros should be removed, except if the number is zero itself.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: bcabc

Output: abc

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char s[10005], stack[10005];
```

```
    int last[26], used[26] = {0};
```

```
    int top = -1;
```

```
    scanf("%s", s);
```

```
    int n = strlen(s);
```

```
    for (int i = 0; i < n; i++)
```

```
        last[s[i] - 'a'] = i;
```

```
    for (int i = 0; i < n; i++) {
```

```
        char c = s[i];
```

```
        if (used[c - 'a'])
```

```
            continue;
```

```
        while (top >= 0 && stack[top] > c && last[stack[top] - 'a'] > i) {
```

```
            used[stack[top] - 'a'] = 0;
```



```
    top--;  
}  
  
    stack[++top] = c;  
    used[c - 'a'] = 1;  
}  
  
    stack[top + 1] = '\0';  
    printf("%s", stack);  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Given the arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the station so that no train is kept waiting. Consider that all the trains arrive on the same day and leave on the same day.

Arrival and departure times can never be the same for a train, but we can have the arrival time of one train equal to the departure time of the other. At any given instance, the same platform cannot be used for both the departure of a train and the arrival of another train. In such cases, we need different platforms.

Example 1

Input:

N=6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}, dep[] = {0910, 1200, 1120, 1130, 1900, 2000}

Output: 3

Explanation: A minimum of three platforms are required to safely arrive

and depart all trains.

Input Format

The first line consists of an integer N, which represents the total number of trains.

The second line consists of N space-separated integers representing the arrival time.

The third line consists of N space-separated integers representing the departure time.

Output Format

The output displays an integer which is the minimum number of platforms required to safely arrive and depart the train.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

0900 0940 0950 1100 1500 1800

0910 1200 1120 1130 1900 2000

Output: 3

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
void sort(int arr[], int n) {  
    for (int i = 0; i < n-1; i++)  
        for (int j = i+1; j < n; j++)  
            if (arr[i] > arr[j]) {  
                int t = arr[i];  
                arr[i] = arr[j];  
                arr[j] = t;  
            }  
}
```

```

int main() {
    int n;
    scanf("%d", &n);

    int arr[20], dep[20];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &dep[i]);

    sort(arr, n);
    sort(dep, n);

    int i = 0, j = 0;
    int platforms = 0, maxPlat = 0;

    while (i < n && j < n) {
        if (arr[i] <= dep[j]) {
            platforms++;
            i++;
            if (platforms > maxPlat)
                maxPlat = platforms;
        } else {
            platforms--;
            j++;
        }
    }

    printf("%d", maxPlat);

    return 0;
}

```

Status : Correct

Marks : 10/10